

Neovim Complete Field Guide

A practical, searchable map of the Vim language and the mappings declared in this checkout. Every custom row below is rebuilt from the current Lua configuration—new or changed keys replace the old documentation.

1 Find

Space f f opens files; Space f g searches project text.

2 Move

w / b move by word; / then n searches; gg / G jump in a file.

3 Change

Use operator + object: ciw changes a word, daw deletes a word.

4 Check

Space l f formats an LSP buffer; :w saves; Space l g opens LazyGit.

How to read this: Space f f means press Space, then f, then f. Context labels identify keys that only appear in Markdown, NvimTree, dashboard, or LSP-attached buffers.

Runtime audit skipped: 0 described runtime mappings observed. The source inventory is the published command truth; runtime data is used as a cross-check.

Character, line & screen

<code>h j k l</code>	Left, down, up, right.
<code>0 ^ \$</code>	Line start / first nonblank / line end.
<code>H M L</code>	Top / middle / bottom visible screen line.
<code>Ctrl-u Ctrl-d</code>	Half-page up / down.
<code>zt zz zb</code>	Place cursor line at top / center / bottom.

Words, paragraphs & file

<code>w b e</code>	Next word start / previous word start / word end.
<code>W B E</code>	Same by whitespace-separated WORD.
<code>{ }</code>	Previous / next paragraph.
<code>gg G</code>	First / last line of the file.
<code>:42 42G</code>	Go to line 42.
<code>%</code>	Jump between matching delimiters.
<code>Ctrl-o Ctrl-i</code>	Older / newer jump location.

Find on one line

<code>f x F x</code>	Find character x forward / backward; land on it.
<code>t x T x</code>	Move until just before x, forward / backward.
<code>; ,</code>	Repeat the latest find in the same / opposite direction.
<code>* #</code>	Search word under cursor forward / backward.

Counts repeat the next command: 3w moves three words; 5dd deletes five lines; d2j deletes the current plus next two lines; 2ci(changes inside the next two parenthesized areas.

Your current configuration

Key	Action	Context
<code><F2></code>	Toggle line numbers	Global
<code><F3></code>	Toggle relative line numbers	Global
<code>Space ?</code>	Buffer Local Keymaps (which-key)	Global
<code>Space nh</code>	Clear search highlights	Global

Operators: the verbs

d	Delete (also copies to a register).
c	Change: delete, then enter Insert mode.
y	Yank: copy to a register.
> <	Indent right / left; add = to auto-indent.
g~ gu gU	Toggle case / lower / upper.
=	Auto-format indentation using filetype rules.

Text objects: the nouns

iw aw	Inner word / word including surrounding space.
i" a"	Inside quotes / quotes plus contents.
i(i[i{	Inside parentheses / brackets / braces.
a(a[a{	Object including delimiters.
ip ap	Inner paragraph / paragraph plus blank space.

Insert, paste & repeat

i I a A	Insert at cursor / first text; append / line end.
o O	Open a line below / above and insert.
p P	Paste after / before cursor or line.
u Ctrl-r	Undo / redo.
.	Repeat the last change.

Read keys as a sentence: dw deletes to the next word; ci" changes inside quotes; ya(copies parentheses and contents. Fast line forms: dd, cc, yy, >>.

Your current configuration

Key	Action	Context
Space ua	Toggle autopairs	Global
Space ui	Toggle indent guides	Global
Space us	Toggle indent scope	Global

Search, replace, save & get unstuck

Search within this file

<code>/text</code> <code>Enter</code>	Search forward /
<code>?text</code> <code>Enter</code>	backward.
<code>n</code> <code>N</code>	Next / previous match.
<code>:noh</code>	Clear highlighting, not the search pattern.
<code>/\cword</code> <code>/\Cword</code>	Force case-insensitive / case-sensitive search.
<code>/\Vliteral</code>	Search punctuation literally.
<code>q/</code>	Open editable search history.

Substitute

<code>:s/old/new/</code>	Replace first match on this line.
<code>:s/old/new/g</code>	Replace every match on this line.
<code>:%s/old/new/gc</code>	Whole file: replace all and confirm each.
<code>:'<,'>s/old/new/g</code>	Replace only the current visual selection.
<code>&</code> <code>:%&</code>	Repeat the latest substitute on this line / whole file.

Help & recovery

<code>:help {topic}</code>	Open built-in documentation.
<code>:map</code> <code>:nmap</code> <code>:imap</code>	Inspect mappings by mode.
<code>:checkhealth</code>	Diagnose Neovim, providers, and plugins.
<code>:messages</code>	Show recent notifications and errors.
<code>:w</code> <code>:q</code> <code>:wq</code> <code>:q!</code>	Save / quit / save+quit / force quit.

Your setup uses ignorecase + smartcase: lowercase searches ignore case; adding any capital makes the search case-sensitive.

Your current configuration

Key	Action	Context
<code>[t</code>	Previous todo comment	Global
<code>]t</code>	Next todo comment	Global
<code>Space f/</code>	Find in buffer	Global
<code>Space fb</code>	Find buffers	Global
<code>Space fd</code>	Find diagnostics	Global
<code>Space ff</code>	Find files	Global

Neovim Field Guide • Ctrl-F searchable • source-driven regeneration

Buffers

<code>:ls</code> <code>:buffers</code>	List open buffers.
<code>:b name</code>	Switch by partial buffer name.
<code>:bnext</code> <code>:bprev</code>	Next / previous buffer.
<code>:bdelete</code>	Close buffer without necessarily closing its window.

Windows

<code>:split</code> <code>:vsplit</code>	Horizontal / vertical split.
<code>Ctrl-w h/j/k/l</code>	Move focus between splits.
<code>Ctrl-w c</code> <code>Ctrl-w o</code>	Close current / keep only current window.
<code>Ctrl-w =</code>	Equalize split sizes.

Tabs, folds & lists

<code>:tabnew</code> <code>gt</code> <code>gT</code>	New tab page / next / previous tab.
<code>za</code> <code>zR</code> <code>zM</code>	Toggle fold / open all / close all.
<code>:copen</code> <code>:cnext</code>	Open quickfix / next project-wide result.
<code>:lopen</code>	Open location list for this window.

Use Space f f for file names, Space f g for project text, / for the current file, and Space f / for fuzzy filtering inside the current buffer.

Your current configuration

Key	Action	Context
<code>Shift-h</code>	Previous buffer	Global
<code>Shift-l</code>	Next buffer	Global
<code>Space bd</code>	Close buffer	Global
<code>Space bp</code>	Pick buffer	Global
<code>Space wr</code>	Find session	Global
<code>Space ws</code>	Save session	Global

Choose the right Git tool

Space h ...	Act on one changed hunk: stage, reset, preview, blame, diff.
Space g ...	Find Git information through Telescope.
Space l g	Open the full LazyGit workflow.

Diagnostics & language features

K	Show the LSP signature, documentation, and type information for the symbol under the cursor.
gd	Jump to the LSP definition of the symbol under the cursor.
gi	Jump to the concrete implementation of the symbol under the cursor.
[d]]d	Previous / next diagnostic; read its message.
Space l d	Show the full diagnostic at the cursor.
Space l a	Open available LSP fixes and refactors for the code under the cursor.
Space l f	Format the current LSP buffer; Markdown can use Prettier.
Space l i	Toggle inlay hints when the server supports them.

Completion & snippets

Ctrl-Space	Manually trigger LSP completion; select an item to show its documentation popup.
Tab Shift-Tab	Jump forward / backward through an active snippet; otherwise type a tab.
Ctrl-n Ctrl-p	Next / previous built-in completion candidate.
Ctrl-f Ctrl-b	Scroll the selected completion item's documentation.
Ctrl-y Ctrl-e	Accept / cancel a built-in completion candidate.
Ctrl-x Ctrl-o	Request omnifunc completion where supported.

Diagnostic means an editor-reported error, warning, or hint. LSP is the language-aware engine that supplies diagnostics, formatting, completion, navigation, and more.

Your current configuration

Key	Action	Context
[h	Previous hunk	Global
]h	Next hunk	Global
ih	Select hunk	Global
ih	Select hunk	Global
Space hB	Toggle line blame	Global
Space hb	Blame line	Global

Markdown workflow

Preview	Rendered, hybrid, or split preview helps read while editing; Fenced Mermaid blocks use the browser's Mermaid.js bundle and refresh smoothly inline; Space mv opens an interactive browser viewer for the current diagram (drag to pan, wheel to zoom, fit/reset/fullscreen, export SVG, PNG, JPEG, or WebP, toggle light/dark with t). Space mD toggles the inline terminal theme; Space mb opens the full Markdown document.
Links	Follow the current link or move through nearby links.
Tables & tasks	Format a table or toggle a task checkbox.
Writing hygiene	Project spell words, format-on-save, table mode, and linting are configured per project.

Spelling suggestions & dictionary

z=	Show spelling suggestions for the misspelled word under the cursor; choose a number and press Enter.
]s [s	Jump to the next / previous misspelling.
zg	Add the word under the cursor to the first file in 'spellfile'; in Markdown buffers here, that is the project dictionary.
Space mw Space mW	Add one / all misspellings to the project dictionary; Space mw is equivalent to zg in Markdown buffers.
:setlocal spell? :setlocal spell	Check whether spelling is active / enable it in this buffer.

Lint diagnostics & fixes

Space md	Run markdownlint-cli2 manually in Markdown or R Markdown.
Save	markdownlint-cli2 runs automatically after a Markdown or R Markdown write.
]d [d	Jump to the next / previous lint or LSP diagnostic.
Space ld	Show the complete diagnostic at the cursor.
Space lf	Format with Prettier; it is not a markdownlint fix-all command.
Fix a lint warning	Edit the Markdown rule violation, then save to lint again; nvim-lint reports findings but does not apply markdownlint fixes here.

TODO discovery

TODO FIXME Quickfix location list	Use clear comment markers so they are searchable.
FIX HACK	Send discovered TODOs to a navigable list.

Autopairs

Matching delimiters are configurable and intentionally disabled in picker/commit contexts.

Markdown buffers automatically enable US English spell checking, soft wrapping, line breaks, and break indentation in this setup. Markdown linting is separate: markdownlint-cli2 supplies diagnostics, while Prettier formats the buffer.

Your current configuration

Key	Action	Context
Space ma	Toggle format on save	Global
Space mb	Browser preview (Mermaid)	Global
Space mc	Toggle task	Global
Space mc	Toggle task	Global
Space md	Lint Markdown	Global
Space mF	Unfold section	Global

Registers, visual mode & block edits

Registers: where copied text lives

"0	Latest yank; "Op pastes the last copied text.
"+	System clipboard in this setup.
"a	Named register: "ayy stores a line; "ap pastes it.
"_	Black-hole register: delete without replacing the clipboard.
:reg	Inspect copied text and registers.

Visual mode

v V	Select characters / whole lines, then use an operator.
Ctrl-v	Select a rectangle; I inserts on every selected line after Esc.
gv	Reselect the previous visual selection.
~	Swap selected text's case.
J gJ	Join lines with / without adding a space.

Your configuration uses unnamedplus, so ordinary yank, delete, and paste already share the system clipboard.

Your current configuration

Key	Action	Context
Space ua	Toggle autopairs	Global
Space ui	Toggle indent guides	Global
Space us	Toggle indent scope	Global

Sessions, dashboard & personal display

Sessions

Find session	Restore a saved workspace.
Save session	Persist the current layout and buffers.
What is saved	Buffers, layout, current directory, terminals, folds, tab pages, and local options.

Dashboard

New file	Start an empty editing buffer.
File recent text projects	Start common Telescope workflows from the dashboard.
Explorer sessions LazyGit	Open a workspace tool before entering an editing buffer.
Plugin manager quit	Maintain plugins or exit Neovim.

Personal display

F2 F3	Toggle absolute / relative line numbers.
Clear highlighting	Remove the visual highlight of the last search.
Indent guides	Toggle indentation and its scope highlight.
Autopairs	Toggle automatic bracket and quote pairing.

Dashboard shortcuts are intentionally context-labeled in the generated appendix: use them from the Alpha start screen, not from an editing buffer.

Your current configuration

Key	Action	Context
e	Start a new empty file	Dashboard buffer
f f	Find files	Dashboard buffer
f k	Find keymaps	Dashboard buffer
f p	Switch projects	Dashboard buffer
f r	Open recent files	Dashboard buffer
f s	Search project text	Dashboard buffer

Testing, debugging & code navigation

Testing loop

Run nearest test	Start with the test closest to the cursor.
Run file suite	Widen scope after the nearest test is useful.
Test output	Open output when a result needs diagnosis.
Toggle summary	Keep test state visible while editing.

Debugging loop

Toggle breakpoint	Mark the next place execution should pause.
Continue	Run to the next breakpoint.
Step over into out	Move through execution without losing the current state.
Inspect UI	Open debugger panels for variables, stack, and console.

Code navigation

Definition references	Jump to where a symbol is defined or used.
Rename	Apply a language-aware rename through the LSP.
Code action	See available fixes and refactors.
Diagnostics	Move through editor-reported problems before changing code.

The complete appendix has the exact current keys. This page teaches the workflow first so it remains useful even when those keys evolve.

Your current configuration

Key	Action	Context
Space tA	Fallback: run test suite	Global
Space ta	Run test suite	Global
Space td	Debug nearest test	Global
Space tf	Run current file	Global
Space tn	Run nearest test	Global
Space to	Show test output	Global

Ctrl+F index

Search this PDF for an exact key string or an ordinary word. The guide repeats literal keys in the teaching sections and keeps a complete generated appendix after them.

Files & project navigation

find files, project text, recent files, buffers, help, keymaps, explorer

Editing

change, delete, copy, paste, repeat, register, text object, visual block

Commands & Lua functions

Ex command, command mode, set, source, lua, function, vim.api, vim.fn, vim.lsp, vim.diagnostic, plugin API

Code help

diagnostic, format, completion, inlay hint, LSP, snippet, code action, rename, definition

Git

hunk, stage, reset, blame, diff, branch, commit, LazyGit

Writing, spelling & lint fixes

Markdown, link, preview, table, spellcheck, misspelled word, spelling suggestion, dictionary, TODO, lint, markdownlint, diagnostic, warning, fix, format, Prettier

Discover anything else: Space f k opens searchable active keymaps. :help keyword explains built-ins; :map, :nmap, and :imap inspect mappings directly; :checkhealth diagnoses the setup.

Configured Tooling

Declared integrations are listed even when they do not expose a direct user mapping.

Tool / plugin	What it enables	Configuration source
DAP adapter: codelldb	Connects Neovim's debugging controls to this language/runtime.	dap.lua
DAP adapter: debugpy	Connects Neovim's debugging controls to this language/runtime.	dap.lua
DAP adapter: delve	Connects Neovim's debugging controls to this language/runtime.	dap.lua
DAP adapter: netcoredbg	Connects Neovim's debugging controls to this language/runtime.	dap.lua
DAP adapter: python	Connects Neovim's debugging controls to this language/runtime.	dap.lua
Executable: codelldb	External command used by a configured editor integration.	dap.lua
Executable: dlv	External command used by a configured editor integration.	dap.lua
Executable: magick	External command used by a configured editor integration.	mermaid_preview.lua
Executable: magick	External command used by a configured editor integration.	image.lua
Executable: nvr	External command used by a configured editor integration.	set.lua
Executable: python	External command used by a configured editor integration.	dap.lua

Configured Tooling - part 2 of 8

Declared integrations are listed even when they do not expose a direct user mapping.

Tool / plugin	What it enables	Configuration source
Executable: python3	External command used by a configured editor integration.	dap.lua
Executable: uv	External command used by a configured editor integration.	neotest.lua
Formatter: prettier	Rewrites source into the project's configured formatting style.	markdown.lua
Linters: markdownlint-cli2	Checks source for style, correctness, or writing issues.	markdown.lua
LSP server: basedpyright	Adds language-aware diagnostics, navigation, completion, formatting, and refactors when attached.	lsp.lua
LSP server: jsonls	Adds language-aware diagnostics, navigation, completion, formatting, and refactors when attached.	lsp.lua
LSP server: lua_ls	Adds language-aware diagnostics, navigation, completion, formatting, and refactors when attached.	lsp.lua
LSP server: roslyn_ls	Adds language-aware diagnostics, navigation, completion, formatting, and refactors when attached.	lsp.lua
LSP server: yamlls	Adds language-aware diagnostics, navigation, completion, formatting, and refactors when attached.	lsp.lua
Plugin: 3rd/image.nvim	Display supported images inline in Neovim when the terminal graphics backend is available.	image.lua
Plugin: akinsho/bufferline.nvim	Show open buffers as a navigable tab-like strip.	bufferlines.lua

Configured Tooling - part 3 of 8

Declared integrations are listed even when they do not expose a direct user mapping.

Tool / plugin	What it enables	Configuration source
Plugin: <code>akinsho/neotest-go</code>	Run tests, inspect output, and debug a test through DAP.	<code>neotest.lua</code>
Plugin: <code>antoinemadec/FixCursorHold.nvim</code>	Configured integration; use its matching mapping or command in this guide.	<code>neotest.lua</code>
Plugin: <code>b0o/SchemaStore.nvim</code>	Configured integration; use its matching mapping or command in this guide.	<code>lsp.lua</code>
Plugin: <code>CollinRobison/neotest-dotnet</code>	Run tests, inspect output, and debug a test through DAP.	<code>neotest.lua</code>
Plugin: <code>folke/todo-comments.nvim</code>	Recognize TODO-style annotations and make them searchable through lists and Telescope.	<code>todo-comments.lua</code>
Plugin: <code>folke/tokyonight.nvim</code>	Configured integration; use its matching mapping or command in this guide.	<code>tokyonight.lua</code>
Plugin: <code>folke/which-key.nvim</code>	Display available keybinding continuations after a key prefix.	<code>which-key.lua</code>
Plugin: <code>goolord/alpha-nvim</code>	Show a start dashboard with shortcuts for files, projects, sessions, Git, and plugins.	<code>alpha.lua</code>
Plugin: <code>haydenmeade/neotest-jest</code>	Run tests, inspect output, and debug a test through DAP.	<code>neotest.lua</code>
Plugin: <code>iamcco/markdown-preview.nvim</code>	Configured integration; use its matching mapping or command in this guide.	<code>markdown.lua</code>
Plugin: <code>image/jpeg</code>	Configured integration; use its matching mapping or command in this guide.	<code>mermaid_preview.lua</code>

Configured Tooling - part 4 of 8

Declared integrations are listed even when they do not expose a direct user mapping.

Tool / plugin	What it enables	Configuration source
Plugin: <code>image/png</code>	Configured integration; use its matching mapping or command in this guide.	<code>mermaid_preview.lua</code>
Plugin: <code>image/webp</code>	Configured integration; use its matching mapping or command in this guide.	<code>mermaid_preview.lua</code>
Plugin: <code>jakewvincent/mkdnflow.nvim</code>	Follow links, manage tasks, tables, headings, and Markdown navigation.	<code>markdown.lua</code>
Plugin: <code>jay-babu/mason-nvim-dap.nvim</code>	Pause a program, step through execution, and inspect debugger state.	<code>mason-tools.lua</code>
Plugin: <code>kdheepak/lazygit.nvim</code>	Open a terminal Git interface for status, commits, branches, and logs.	<code>lazygit.lua</code>
Plugin: <code>lewis6991/git signs.nvim</code>	Navigate, preview, stage, reset, and blame changed Git hunks.	<code>git signs.lua</code>
Plugin: <code>lukas-reineke/indent-blankline.nvim</code>	Draw indentation guides and optionally highlight the current scope.	<code>indent-blankline.lua</code>
Plugin: <code>marilari88/neotest-vitest</code>	Run tests, inspect output, and debug a test through DAP.	<code>neotest.lua</code>
Plugin: <code>mason-org/mason-lspconfig.nvim</code>	Install and manage external language servers, formatters, linters, and debug adapters.	<code>lsp.lua</code>
Plugin: <code>mason-org/mason-lspconfig.nvim</code>	Install and manage external language servers, formatters, linters, and debug adapters.	<code>mason-tools.lua</code>
Plugin: <code>mason-org/mason.nvim</code>	Install and manage external language servers, formatters, linters, and debug adapters.	<code>mason-tools.lua</code>

Configured Tooling - part 5 of 8

Declared integrations are listed even when they do not expose a direct user mapping.

Tool / plugin	What it enables	Configuration source
Plugin: <code>mfussenegger/nvim-dap</code>	Pause a program, step through execution, and inspect debugger state.	<code>dap.lua</code>
Plugin: <code>mfussenegger/nvim-lint</code>	Run configured linters and publish their findings as diagnostics; this setup does not apply markdownlint fixes automatically.	<code>markdown.lua</code>
Plugin: <code>neovim/nvim-lspconfig</code>	Configured integration; use its matching mapping or command in this guide.	<code>lsp.lua</code>
Plugin: <code>nvim-lua/plenary.nvim</code>	Configured integration; use its matching mapping or command in this guide.	<code>todo-comments.lua</code>
Plugin: <code>nvim-lua/plenary.nvim</code>	Configured integration; use its matching mapping or command in this guide.	<code>lazygit.lua</code>
Plugin: <code>nvim-lua/plenary.nvim</code>	Configured integration; use its matching mapping or command in this guide.	<code>telescope.lua</code>
Plugin: <code>nvim-lualine/lualine.nvim</code>	Show file, Git, diagnostic, mode, and status information in the statusline.	<code>lualine.lua</code>
Plugin: <code>nvim-mini/mini.icons</code>	Provide compact file-type icons to Neovim UI integrations.	<code>mini-icon.lua</code>
Plugin: <code>nvim-neotest/neotest</code>	Run tests, inspect output, and debug a test through DAP.	<code>neotest.lua</code>
Plugin: <code>nvim-neotest/neotest-python</code>	Run tests, inspect output, and debug a test through DAP.	<code>neotest.lua</code>
Plugin: <code>nvim-neotest/neotest-vim-test</code>	Run tests, inspect output, and debug a test through DAP.	<code>neotest.lua</code>

Configured Tooling - part 6 of 8

Declared integrations are listed even when they do not expose a direct user mapping.

Tool / plugin	What it enables	Configuration source
Plugin: nvim-neotest/nvim-nio	Run tests, inspect output, and debug a test through DAP.	dap.lua
Plugin: nvim-neotest/nvim-nio	Run tests, inspect output, and debug a test through DAP.	neotest.lua
Plugin: nvim-telescope/telescope-fzf-native.nvim	Search files, text, buffers, help, keymaps, and Git information.	telescope.lua
Plugin: nvim-telescope/telescope-project.nvim	Search files, text, buffers, help, keymaps, and Git information.	telescope.lua
Plugin: nvim-telescope/telescope.nvim	Search files, text, buffers, help, keymaps, and Git information.	telescope.lua
Plugin: nvim-tree/nvim-tree.lua	Browse files and reveal the current file in a tree.	nvimtree.lua
Plugin: nvim-tree/nvim-web-devicons	Browse files and reveal the current file in a tree.	luafile.lua
Plugin: nvim-tree/nvim-web-devicons	Browse files and reveal the current file in a tree.	nvimtree.lua
Plugin: nvim-tree/nvim-web-devicons	Browse files and reveal the current file in a tree.	nvim-web-devicons.lua
Plugin: nvim-tree/nvim-web-devicons	Browse files and reveal the current file in a tree.	bufferlines.lua
Plugin: nvim-treesitter/nvim-treesitter	Browse files and reveal the current file in a tree.	nvim-treesitter.lua

Configured Tooling - part 7 of 8

Declared integrations are listed even when they do not expose a direct user mapping.

Tool / plugin	What it enables	Configuration source
Plugin: nvim-treesitter/nvim-treesitter	Browse files and reveal the current file in a tree.	image.lua
Plugin: orjangj/neotest-ctest	Run tests, inspect output, and debug a test through DAP.	neotest.lua
Plugin: OXY2DEV/markview.nvim	Render Markdown while you edit it.	markdown.lua
Plugin: rcarriga/nvim-dap-ui	Pause a program, step through execution, and inspect debugger state.	dap.lua
Plugin: rmagatti/auto-session	Save and restore working layouts and open buffers.	auto-session.lua
Plugin: rouge8/neotest-rust	Run tests, inspect output, and debug a test through DAP.	neotest.lua
Plugin: stevearc/conform.nvim	Configured integration; use its matching mapping or command in this guide.	markdown.lua
Plugin: text/html	Configured integration; use its matching mapping or command in this guide.	mermaid_preview.lua
Plugin: textDocument/codeLens	Configured integration; use its matching mapping or command in this guide.	lsp.lua
Plugin: textDocument/completion	Configured integration; use its matching mapping or command in this guide.	lsp.lua
Plugin: textDocument/definition	Configured integration; use its matching mapping or command in this guide.	lsp.lua

Configured Tooling - part 8 of 8

Declared integrations are listed even when they do not expose a direct user mapping.

Tool / plugin	What it enables	Configuration source
Plugin: <code>textDocument/hover</code>	Configured integration; use its matching mapping or command in this guide.	<code>lsp.lua</code>
Plugin: <code>textDocument/implementation</code>	Configured integration; use its matching mapping or command in this guide.	<code>lsp.lua</code>
Plugin: <code>textDocument/inlayHint</code>	Configured integration; use its matching mapping or command in this guide.	<code>lsp.lua</code>
Plugin: <code>theHamsta/nvim-dap-virtual-text</code>	Pause a program, step through execution, and inspect debugger state.	<code>dap.lua</code>
Plugin: <code>vague-theme/vague.nvim</code>	Configured integration; use its matching mapping or command in this guide.	<code>vague.lua</code>
Plugin: <code>vim-test/vim-test</code>	Configured integration; use its matching mapping or command in this guide.	<code>neotest.lua</code>
Plugin: <code>windwp/nvim-autopairs</code>	Automatically insert and manage matching quotes, brackets, and parentheses while typing.	<code>autopairs.lua</code>
Plugin: <code>windwp/nvim-ts-autotag</code>	Configured integration; use its matching mapping or command in this guide.	<code>nvim-treesitter.lua</code>

Essential Vim keys

The key language behind Normal, Visual, and Insert mode editing.

Area	Command / function	What it does
Move precisely, then act	h j k l	Move left / down / up / right.
Move precisely, then act	w b e	Next word start / previous word start / word end.
Move precisely, then act	0 ^ \$	Line start / first nonblank / line end.
Move precisely, then act	gg G	First / last line of the file.
Move precisely, then act	f{char} ; ,	Find on a line; repeat forward / backward.
Move precisely, then act	/text n N	Search; next / previous result.
Move precisely, then act	Ctrl-o Ctrl-i	Older / newer jump location.
Edit with operators & text objects	d c y	Delete / change / yank; pair with a motion or object.
Edit with operators & text objects	ciw daw yi"	Change inner word / delete a word / yank inside quotes.
Edit with operators & text objects	i(a(ip ap	Inner / around parentheses; inner / around paragraph.
Edit with operators & text objects	dd cc yy	Delete / change / yank a whole line.
Edit with operators & text objects	p P	Paste after / before the cursor or line.

Essential Vim keys · part 2 of 2

The key language behind Normal, Visual, and Insert mode editing.

Area	Command / function	What it does
Edit with operators & text objects	u Ctrl-r .	Undo / redo / repeat the last change.
Files, windows & working memory	:e {file} :w :q	Open file / save / quit.
Files, windows & working memory	:b {buffer}	Switch buffer by name or number.
Files, windows & working memory	Ctrl-w h j k l	Move between split windows.
Files, windows & working memory	za zc zo	Toggle / close / open a fold.
Files, windows & working memory	m{a} '{a}	Set a mark / jump to its line.
Files, windows & working memory	q{a} q @a @@	Record macro / stop / run / repeat macro.
Select, complete & inspect	v V Ctrl-v	Character / line / block selection.
Select, complete & inspect	]s [s z=	Next misspelling / previous / suggestions.
Select, complete & inspect	Ctrl-n Ctrl-p	Built-in next / previous completion candidate.
Select, complete & inspect	Ctrl-y Ctrl-e	Accept / cancel built-in completion.
Select, complete & inspect	:help {topic}	Open built-in help; use Ctrl-] to follow tags.

Common Ex & plugin commands

Commands are entered after `:` in Normal mode. Arguments in braces are placeholders; the final entries are plugin commands from this setup.

Area	Command / function	What it does
Help & diagnosis	<code>:help {topic}</code>	Open built-in documentation; use <code>:help user-manual</code> for the full guide.
Help & diagnosis	<code>:helpgrep {pattern}</code>	Search all runtime help tags and text.
Help & diagnosis	<code>:checkhealth</code>	Run health checks for Neovim, providers, and plugins.
Help & diagnosis	<code>:messages</code>	Show recent notifications, errors, and command output.
Help & diagnosis	<code>:version</code>	Show the Neovim version and compiled features.
Files & buffers	<code>:edit {file}</code>	Open a file in the current window.
Files & buffers	<code>:enew</code>	Create a new empty buffer.
Files & buffers	<code>:write</code> / <code>:update</code>	Save the buffer; <code>:update</code> writes only when it changed.
Files & buffers	<code>:saveas {file}</code>	Write the buffer to a new file name.
Files & buffers	<code>:quit</code> / <code>:qall</code>	Close the current window / quit all of Neovim.
Files & buffers	<code>:writequit</code> / <code>:xit</code>	Save and close the current window.
Files & buffers	<code>:buffers</code> / <code>:buffer {name}</code>	List buffers / switch to a buffer by name or number.
Files & buffers	<code>:bnext</code> / <code>:bprevious</code>	Switch to the next / previous buffer.
Files & buffers	<code>:bdelete</code>	Unload the current buffer without necessarily closing its window.

Common Ex & plugin commands · part 2 of 3

Commands are entered after `:` in Normal mode. Arguments in braces are placeholders; the final entries are plugin commands from this setup.

Area	Command / function	What it does
Windows & tabs	<code>:split / :vsplit {file}</code>	Open a horizontal / vertical split, optionally with a file.
Windows & tabs	<code>:close / :only</code>	Close the current window / keep only the current window.
Windows & tabs	<code>:resize {height} / :vertical resize {width}</code>	Resize the current split.
Windows & tabs	<code>:tabnew {file}</code>	Open a new tab page, optionally with a file.
Windows & tabs	<code>:tabclose / :tabonly</code>	Close the current tab / keep only the current tab.
Windows & tabs	<code>:tabnext / :tabprevious</code>	Switch to the next / previous tab page.
Settings & mappings	<code>:set {option}?</code>	Inspect an option's current value.
Settings & mappings	<code>:set {option} / :set no{option}</code>	Enable / disable a boolean option.
Settings & mappings	<code>:setlocal {option}</code>	Change an option only for the current buffer or window.
Settings & mappings	<code>:verbose set {option}?</code>	Show an option's value and where it was last set.
Settings & mappings	<code>:map / :nmap / :imap</code>	List mappings, or mappings for Normal / Insert mode.
Settings & mappings	<code>:verbose nmap {key}</code>	Show a mapping and the file or plugin that defined it.
Settings & mappings	<code>:command</code>	List user-defined Ex commands.
Configuration	<code>:source \$MYVIMRC</code>	Reload the current Neovim configuration file.

Common Ex & plugin commands - part 3 of 3

Commands are entered after `:` in Normal mode. Arguments in braces are placeholders; the final entries are plugin commands from this setup.

Area	Command / function	What it does
Configuration	<code>:lua {expression}</code>	Evaluate a Lua expression or call a Lua function.
Search & lists	<code>:nohlsearch</code>	Clear search highlighting without changing the search pattern.
Search & lists	<code>:copen</code> / <code>:cclose</code>	Open / close the quickfix list.
Search & lists	<code>:cnext</code> / <code>:cprevious</code>	Jump to the next / previous quickfix item.
Search & lists	<code>:lopen</code> / <code>:lclose</code>	Open / close the current window's location list.
Search & lists	<code>:lnext</code> / <code>:lprevious</code>	Jump to the next / previous location-list item.
Search & lists	<code>:vimgrep</code> / <code>:vimgrepadd</code>	Search files with Vim's regex engine and populate quickfix.
Editing	<code>:normal {keys}</code>	Execute Normal-mode keys on the current line or selected range.
Editing	<code>:undo</code> / <code>:redo</code>	Undo / redo a change from the command line.
Editing	<code>:retab</code>	Replace tab characters according to the current 'tabstop' settings.
Editing	<code>:sort [range]</code>	Sort lines in the current or selected range.
Plugins	<code>:Lazy</code>	Open Lazy.nvim's plugin manager.
Plugins	<code>:Mason</code>	Open Mason's package manager for LSP servers and tools.
Plugins	<code>:Telescope {picker}</code>	Open a Telescope picker such as <code>find_files</code> , <code>live_grep</code> , <code>buffers</code> , or <code>help_tags</code> .

Neovim Lua functions

Use these inside Lua callbacks or with :lua. Parentheses show a callable example, not a mapping.

Area	Command / function	What it does
Neovim Lua	<code>vim.cmd("{ex-command}")</code>	Run an Ex command from Lua.
Neovim Lua	<code>vim.keymap.set(mode, lhs, rhs, opts)</code>	Create a keymap with a description, buffer, and other options.
Neovim Lua	<code>vim.notify(message, level)</code>	Show an informational, warning, or error notification.
Neovim Lua	<code>vim.schedule(callback)</code>	Defer a callback until it is safe to update the UI.
Buffers & windows	<code>vim.api.nvim_get_current_buf()</code>	Get the current buffer number.
Buffers & windows	<code>vim.api.nvim_buf_get_name(buf)</code>	Get a buffer's full file name.
Buffers & windows	<code>vim.api.nvim_buf_get_lines(buf, start, end, strict)</code>	Read lines from a buffer; line indexes are zero-based.
Buffers & windows	<code>vim.api.nvim_buf_set_lines(buf, start, end, strict, lines)</code>	Replace a range of buffer lines.
Buffers & windows	<code>vim.api.nvim_win_get_cursor(win)</code>	Get a window cursor as { line, column }.
Buffers & windows	<code>vim.api.nvim_win_set_cursor(win, position)</code>	Move a window cursor programmatically.
Autocommands	<code>vim.api.nvim_create_augroup(name, opts)</code>	Create or reset a named autocommand group.
Autocommands	<code>vim.api.nvim_create_autocmd(event, opts)</code>	Run a callback on events such as BufWritePost, FileType, or LspAttach.
Commands	<code>vim.api.nvim_create_user_command(name, callback, opts)</code>	Define a custom :Command that users can run.
Vim functions	<code>vim.fn.expand("%:p")</code>	Expand the current file, cursor word, or another Vim expression.
Vim functions	<code>vim.fn.getcwd()</code>	Return Neovim's current working directory.

Neovim Lua functions · part 2 of 3

Use these inside Lua callbacks or with :lua. Parentheses show a callable example, not a mapping.

Area	Command / function	What it does
Vim functions	<code>vim.fn.executable("tool")</code>	Check whether an executable is available on PATH.
Vim functions	<code>vim.fn.exepath("tool")</code>	Return the resolved path of an executable.
Vim functions	<code>vim.fn.stdpath("config")</code>	Find Neovim's config, data, state, or cache directory.
Vim functions	<code>vim.fn.system({ command, args })</code>	Run an external command and capture its output.
Vim functions	<code>vim.fn.fnameescape(path)</code>	Escape a file name before passing it to an Ex command.
Vim functions	<code>vim.fn.spellbadword()</code>	Return the misspelled word at the cursor and its spelling type.
Filesystem	<code>vim.fs.find(names, opts)</code>	Find files or directories upward or downward from a path.
Filesystem	<code>vim.fs.root(path, markers)</code>	Find a project root using marker files such as .git or pyproject.toml.
Diagnostics	<code>vim.diagnostic.open_float()</code>	Show the complete diagnostic message at the cursor.
Diagnostics	<code>vim.diagnostic.jump({ count = 1, float = true })</code>	Jump to the next or previous diagnostic and optionally show it.
Diagnostics	<code>vim.diagnostic.get(buf, opts)</code>	Read diagnostics for a buffer, namespace, severity, or range.
LSP	<code>vim.lsp.buf.hover()</code>	Request hover documentation for the symbol under the cursor.
LSP	<code>vim.lsp.buf.definition()</code>	Jump to the definition of the symbol under the cursor.
LSP	<code>vim.lsp.buf.references()</code>	List references to the symbol under the cursor.
LSP	<code>vim.lsp.buf.rename()</code>	Rename a symbol through the attached language server.

Neovim Lua functions · part 3 of 3

Use these inside Lua callbacks or with :lua. Parentheses show a callable example, not a mapping.

Area	Command / function	What it does
LSP	<code>vim.lsp.buf.code_action()</code>	Request available fixes and refactors from the language server.
LSP	<code>vim.lsp.buf.format({ async = true })</code>	Format the current buffer through a formatter or LSP.
LSP	<code>vim.lsp.get_clients({ bufnr = buf })</code>	List language-server clients attached to a buffer.
LSP	<code>vim.lsp.enable(servers)</code>	Enable a list of configured language servers.
Completion	<code>vim.lsp.completion.get()</code>	Request completion items at the cursor.
Completion	<code>vim.lsp.inlay_hint.enable(enabled, opts)</code>	Enable or disable inline type and parameter hints.
Configured tools	<code>require("lint").try_lint()</code>	Run the configured linter and publish its findings as diagnostics.
Configured tools	<code>require("conform").format()</code>	Run the configured formatter, such as Prettier for Markdown.
Configured tools	<code>require("telescope.builtin").find_files()</code>	Open Telescope's file finder from Lua.
Configured tools	<code>require("nvim-tree.api").tree.toggle()</code>	Open or close the configured file explorer.
Configured tools	<code>require("gitsigns").stage_hunk()</code>	Stage the Git hunk under the cursor.
Configured tools	<code>require("neotest").run.run()</code>	Run the test nearest the cursor.
Configured tools	<code>require("dap").continue()</code>	Start or continue a debugger session.

Functions used by this configuration

Callable Neovim, Vimscript, and plugin APIs found in the Lua source. These are useful from `:lua` or inside a callback.

Function	What it does	Source
<code>require("collin.mermaid_preview").setup</code>	Call a configured plugin API from Lua.	<code>markdown.lua</code>
<code>require("gitsigns").blame_line</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").diffthis</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").nav_hunk</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").preview_hunk</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").preview_hunk_inline</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").reset_buffer</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").reset_buffer_index</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").reset_hunk</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").select_hunk</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").setloclist</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").setqflist</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").stage_buffer</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").stage_hunk</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").toggle_current_line_blame</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("gitsigns").toggle_word_diff</code>	Call a configured plugin API from Lua.	<code>gitsigns.lua</code>
<code>require("ibl").setup</code>	Call a configured plugin API from Lua.	<code>indent-blankline.lua</code>
<code>require("lazy").load</code>	Call a configured plugin API from Lua.	<code>todo-comments.lua</code>

Functions used by this configuration - part 2 of 7

Callable Neovim, Vimscript, and plugin APIs found in the Lua source. These are useful from `:lua` or inside a callback.

Function	What it does	Source
<code>require("lazy").setup</code>	Call a configured plugin API from Lua.	<code>lazy_init.lua</code>
<code>require("lint").try_lint</code>	Run configured linters and publish their findings as diagnostics.	<code>markdown.lua</code>
<code>require("lualine").setup</code>	Call a configured plugin API from Lua.	<code>lualine.lua</code>
<code>require("markview").commands.Render</code>	Call a configured plugin API from Lua.	<code>markdown.lua</code>
<code>require("mason").setup</code>	Call a configured plugin API from Lua.	<code>mason-tools.lua</code>
<code>require("mason-lspconfig").setup</code>	Call a configured plugin API from Lua.	<code>lsp.lua</code>
<code>require("mason-nvim-dap").setup</code>	Call a configured plugin API from Lua.	<code>mason-tools.lua</code>
<code>require("neotest-ctest").setup</code>	Call a configured plugin API from Lua.	<code>neotest.lua</code>
<code>require("neotest-jest.jest-util").defaultIsTestFile</code>	Call a configured plugin API from Lua.	<code>neotest.lua</code>
<code>require("nvim-autopairs").toggle</code>	Call a configured plugin API from Lua.	<code>autopairs.lua</code>
<code>require("nvim-tree").setup</code>	Call a configured plugin API from Lua.	<code>nvimtree.lua</code>
<code>require("nvim-tree.api").tree.collapse_all</code>	Call a configured plugin API from Lua.	<code>nvimtree.lua</code>
<code>require("nvim-tree.api").tree.expand_all</code>	Call a configured plugin API from Lua.	<code>nvimtree.lua</code>
<code>require("nvim-tree.api").tree.find_file</code>	Call a configured plugin API from Lua.	<code>nvimtree.lua</code>
<code>require("nvim-tree.api").tree.focus</code>	Call a configured plugin API from Lua.	<code>nvimtree.lua</code>
<code>require("nvim-tree.api").tree.reload</code>	Call a configured plugin API from Lua.	<code>nvimtree.lua</code>
<code>require("nvim-tree.api").tree.toggle</code>	Call a configured plugin API from Lua.	<code>nvimtree.lua</code>
<code>require("nvim-ts-autotag").setup</code>	Call a configured plugin API from Lua.	<code>nvim-treesitter.lua</code>

Functions used by this configuration - part 3 of 7

Callable Neovim, Vimscript, and plugin APIs found in the Lua source. These are useful from `:lua` or inside a callback.

Function	What it does	Source
<code>require("telescope").load_extension</code>	Call a configured plugin API from Lua.	<code>todo-comments.lua</code>
<code>require("telescope.actions").close</code>	Call a configured plugin API from Lua.	<code>telescope.lua</code>
<code>require("todo-comments").jump_next</code>	Call a configured plugin API from Lua.	<code>todo-comments.lua</code>
<code>require("todo-comments").jump_prev</code>	Call a configured plugin API from Lua.	<code>todo-comments.lua</code>
<code>require("vague").get_palette</code>	Call a configured plugin API from Lua.	<code>indent-blankline.lua</code>
<code>require("which-key").show</code>	Call a configured plugin API from Lua.	<code>which-key.lua</code>
<code>vim.api.nvim_buf_get_lines</code>	Call a Neovim API function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.api.nvim_buf_get_name</code>	Call a Neovim API function from Lua.	<code>markdown.lua</code>
<code>vim.api.nvim_buf_is_valid</code>	Call a Neovim API function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.api.nvim_create_augroup</code>	Create or reset a named autocommand group.	<code>alpha.lua</code>
<code>vim.api.nvim_create_autocmd</code>	Run a callback when a Neovim event occurs.	<code>alpha.lua</code>
<code>vim.api.nvim_create_user_command</code>	Define a custom Ex command.	<code>mason-tools.lua</code>
<code>vim.api.nvim_get_current_buf</code>	Call a Neovim API function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.api.nvim_get_current_win</code>	Call a Neovim API function from Lua.	<code>markdown.lua</code>
<code>vim.api.nvim_get_runtime_file</code>	Call a Neovim API function from Lua.	<code>lsp.lua</code>
<code>vim.api.nvim_list_wins</code>	Call a Neovim API function from Lua.	<code>image.lua</code>
<code>vim.api.nvim_set_hl</code>	Call a Neovim API function from Lua.	<code>indent-blankline.lua</code>
<code>vim.api.nvim_win_get_buf</code>	Call a Neovim API function from Lua.	<code>image.lua</code>

Functions used by this configuration - part 4 of 7

Callable Neovim, Vimscript, and plugin APIs found in the Lua source. These are useful from `:lua` or inside a callback.

Function	What it does	Source
<code>vim.api.nvim_win_get_config</code>	Call a Neovim API function from Lua.	<code>image.lua</code>
<code>vim.api.nvim_win_get_cursor</code>	Call a Neovim API function from Lua.	<code>markdown.lua</code>
<code>vim.api.nvim_win_get_width</code>	Call a Neovim API function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.api.nvim_win_is_valid</code>	Call a Neovim API function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.api.nvim_win_set_cursor</code>	Call a Neovim API function from Lua.	<code>markdown.lua</code>
<code>vim.base64.encode</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>mermaid_preview.lua</code>
<code>vim.cmd</code>	Run an Ex command from Lua.	<code>markdown.lua</code>
<code>vim.cmd.colorscheme</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>tokyonight.lua</code>
<code>vim.deepcopy</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>nvimtree.lua</code>
<code>vim.defer_fn</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>mermaid_preview.lua</code>
<code>vim.diagnostic.config</code>	Read or navigate editor diagnostics.	<code>lsp.lua</code>
<code>vim.diagnostic.jump</code>	Jump to a previous or next diagnostic.	<code>lsp.lua</code>
<code>vim.filetype.add</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>markdown.lua</code>
<code>vim.fn.bufwinid</code>	Call a built-in Vimscript function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.fn.delete</code>	Call a built-in Vimscript function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.fn.executable</code>	Call a built-in Vimscript function from Lua.	<code>dap.lua</code>
<code>vim.fn.exepath</code>	Call a built-in Vimscript function from Lua.	<code>dap.lua</code>
<code>vim.fn.expand</code>	Call a built-in Vimscript function from Lua.	<code>dap.lua</code>

Functions used by this configuration - part 5 of 7

Callable Neovim, Vimscript, and plugin APIs found in the Lua source. These are useful from `:lua` or inside a callback.

Function	What it does	Source
<code>vim.fn.filereadable</code>	Call a built-in Vimscript function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.fn.fnameescape</code>	Call a built-in Vimscript function from Lua.	<code>telescope.lua</code>
<code>vim.fn.fnamemodify</code>	Call a built-in Vimscript function from Lua.	<code>lua_line.lua</code>
<code>vim.fn.getcwd</code>	Call a built-in Vimscript function from Lua.	<code>dap.lua</code>
<code>vim.fn.getpid</code>	Call a built-in Vimscript function from Lua.	<code>set.lua</code>
<code>vim.fn.input</code>	Call a built-in Vimscript function from Lua.	<code>dap.lua</code>
<code>vim.fn.line</code>	Call a built-in Vimscript function from Lua.	<code>gitsigns.lua</code>
<code>vim.fn.mkdir</code>	Call a built-in Vimscript function from Lua.	<code>markdown.lua</code>
<code>vim.fn.readfile</code>	Call a built-in Vimscript function from Lua.	<code>markdown.lua</code>
<code>vim.fn.rename</code>	Call a built-in Vimscript function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.fn.serverstart</code>	Call a built-in Vimscript function from Lua.	<code>set.lua</code>
<code>vim.fn.sha256</code>	Call a built-in Vimscript function from Lua.	<code>markdown.lua</code>
<code>vim.fn.sign_define</code>	Call a built-in Vimscript function from Lua.	<code>dap.lua</code>
<code>vim.fn.spellbadword</code>	Call a built-in Vimscript function from Lua.	<code>markdown.lua</code>
<code>vim.fn.stdpath</code>	Call a built-in Vimscript function from Lua.	<code>markdown.lua</code>
<code>vim.fn.system</code>	Call a built-in Vimscript function from Lua.	<code>image.lua</code>
<code>vim.fn.tempname</code>	Call a built-in Vimscript function from Lua.	<code>mermaid_preview.lua</code>
<code>vim.fn.writefile</code>	Call a built-in Vimscript function from Lua.	<code>markdown.lua</code>

Functions used by this configuration - part 6 of 7

Callable Neovim, Vimscript, and plugin APIs found in the Lua source. These are useful from `:lua` or inside a callback.

Function	What it does	Source
<code>vim.fs.basename</code>	Inspect project files or roots from Lua.	<code>nvimtree.lua</code>
<code>vim.fs.dirname</code>	Inspect project files or roots from Lua.	<code>markdown.lua</code>
<code>vim.fs.find</code>	Inspect project files or roots from Lua.	<code>dap.lua</code>
<code>vim.fs.joinpath</code>	Inspect project files or roots from Lua.	<code>nvimtree.lua</code>
<code>vim.fs.root</code>	Inspect project files or roots from Lua.	<code>dap.lua</code>
<code>vim.json.decode</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>neotest.lua</code>
<code>vim.json.encode</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>mermaid_preview.lua</code>
<code>vim.keymap.set</code>	Create a keymap with a description and optional buffer or mode.	<code>alpha.lua</code>
<code>vim.list_extend</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>nvimtree.lua</code>
<code>vim.loop.new_pipe</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>image.lua</code>
<code>vim.loop.read_start</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>image.lua</code>
<code>vim.loop.spawn</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>image.lua</code>
<code>vim.lsp.buf.format</code>	Format the current buffer through an LSP or formatter.	<code>lsp.lua</code>
<code>vim.lsp.codelens.enable</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>lsp.lua</code>
<code>vim.lsp.codelens.run</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>lsp.lua</code>
<code>vim.lsp.completion.enable</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>lsp.lua</code>
<code>vim.lsp.completion.get</code>	Request completion items at the cursor.	<code>lsp.lua</code>
<code>vim.lsp.config</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>lsp.lua</code>

Functions used by this configuration - part 7 of 7

Callable Neovim, Vimscript, and plugin APIs found in the Lua source. These are useful from `:lua` or inside a callback.

Function	What it does	Source
<code>vim.lsp.enable</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>lsp.lua</code>
<code>vim.lsp.get_client_by_id</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>lsp.lua</code>
<code>vim.lsp.get_clients</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>lsp.lua</code>
<code>vim.lsp.inlay_hint.enable</code>	Enable or disable inline type and parameter hints.	<code>lsp.lua</code>
<code>vim.lsp.inlay_hint.is_enabled</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>lsp.lua</code>
<code>vim.notify</code>	Show a Neovim notification.	<code>dap.lua</code>
<code>vim.schedule</code>	Defer a callback until it is safe to update the UI.	<code>image.lua</code>
<code>vim.split</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>mermaid_preview.lua</code>
<code>vim.system</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>dap.lua</code>
<code>vim.tbl_contains</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>lsp.lua</code>
<code>vim.tbl_count</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>markdown.lua</code>
<code>vim.tbl_extend</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>markdown.lua</code>
<code>vim.treesitter.language.register</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>nvim-treesitter.lua</code>
<code>vim.trim</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>image.lua</code>
<code>vim.uri_from_fname</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>mermaid_preview.lua</code>
<code>vim.uri_to_fname</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>nvimtree.lua</code>
<code>vim.uv.fs_stat</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>image.lua</code>
<code>vim.uv.new_tcp</code>	Lua/API function used by this configuration. See the source for its arguments.	<code>mermaid_preview.lua</code>

Commands & automatic behavior

Type these after `:` in Normal mode. The tooling inventory explains capabilities that happen automatically or do not have a direct keybinding.

Command	What it does	Configuration source
<code>:AutoSession</code>	Save, search, or restore an editor workspace session.	<code>alpha.lua</code>
<code>:bdelete</code>	Close the current buffer without quitting Neovim.	<code>bufferlines.lua</code>
<code>:BufferLineCycleNext</code>	Select the next buffer in the buffer line.	<code>bufferlines.lua</code>
<code>:BufferLineCyclePrev</code>	Select the previous buffer in the buffer line.	<code>bufferlines.lua</code>
<code>:BufferLinePick</code>	Show letter labels so you can jump directly to an open buffer.	<code>bufferlines.lua</code>
<code>:colorscheme</code>	Select the active colorscheme.	<code>tokyonight.lua</code>
<code>:ene</code>	Open a new empty buffer.	<code>alpha.lua</code>
<code>:IBLToggle</code>	Show or hide indentation guides.	<code>indent-blankline.lua</code>
<code>:IBLToggleScope</code>	Show or hide the current indentation scope highlight.	<code>indent-blankline.lua</code>
<code>:Lazy</code>	Open the plugin manager.	<code>alpha.lua</code>

Commands & automatic behavior - part 2 of 4

Type these after `:` in Normal mode. The tooling inventory explains capabilities that happen automatically or do not have a direct keybinding.

Command	What it does	Configuration source
<code>:LazyGit</code>	Open the terminal Git interface for the repository.	<code>alpha.lua</code>
<code>:LazyGitCurrentFile</code>	Open the terminal Git interface for the repository.	<code>lazygit.lua</code>
<code>:LazyGitFilter</code>	Open the terminal Git interface for the repository.	<code>lazygit.lua</code>
<code>:LazyGitFilterCurrentFile</code>	Open the terminal Git interface for the repository.	<code>lazygit.lua</code>
<code>:MarkdownPreviewToggle</code>	User-facing command declared by this configuration; run it after typing a colon in Normal mode.	<code>markdown.lua</code>
<code>:Markview</code>	Toggle rendered Markdown, hybrid view, or a preview split.	<code>markdown.lua</code>
<code>:MasonInstall</code>	Install one or more Mason packages.	<code>mason-tools.lua</code>
<code>:MasonInstallDebugAdapters</code>	Install the debugger adapters configured for this Neovim setup.	<code>mason-tools.lua</code>
<code>:MermaidBrowserPreview</code>	User-facing command declared by this configuration; run it after typing a colon in Normal mode.	<code>mermaid_preview.lua</code>
<code>:MermaidRefresh</code>	User-facing command declared by this configuration; run it after typing a colon in Normal mode.	<code>mermaid_preview.lua</code>

Commands & automatic behavior · part 3 of 4

Type these after `:` in Normal mode. The tooling inventory explains capabilities that happen automatically or do not have a direct keybinding.

Command	What it does	Configuration source
<code>:MermaidThemeToggle</code>	User-facing command declared by this configuration; run it after typing a colon in Normal mode.	<code>mermaid_preview.lua</code>
<code>:MkdnFoldSection</code>	Fold the Markdown section at the cursor.	<code>markdown.lua</code>
<code>:MkdnFollowLink</code>	Open the Markdown link under the cursor.	<code>markdown.lua</code>
<code>:MkdnNextLink</code>	Jump to the next Markdown link.	<code>markdown.lua</code>
<code>:MkdnPrevLink</code>	Jump to the previous Markdown link.	<code>markdown.lua</code>
<code>:MkdnTableFormat</code>	Align and format the Markdown table at the cursor.	<code>markdown.lua</code>
<code>:MkdnToggleToDo</code>	Toggle the Markdown task checkbox at the cursor.	<code>markdown.lua</code>
<code>:MkdnUnfoldSection</code>	Unfold the Markdown section at the cursor.	<code>markdown.lua</code>
<code>:normal</code>	Execute Normal-mode keys on the current line or range.	<code>markdown.lua</code>
<code>:NvimTreeToggle</code>	Open or close the file explorer.	<code>alpha.lua</code>

Commands & automatic behavior - part 4 of 4

Type these after `:` in Normal mode. The tooling inventory explains capabilities that happen automatically or do not have a direct keybinding.

Command	What it does	Configuration source
<code>:qa</code>	Quit all Neovim windows.	<code>alpha.lua</code>
<code>:silent</code>	User-facing command declared by this configuration; run it after typing a colon in Normal mode.	<code>markdown.lua</code>
<code>:tcd</code>	Change the current tab's working directory.	<code>telescope.lua</code>
<code>:Telescope</code>	Open a searchable picker; the following argument selects files, text, help, Git, or another source.	<code>alpha.lua</code>
<code>:TestFile</code>	Run the configured test target (nearest test, file, or suite).	<code>neotest.lua</code>
<code>:TestNearest</code>	Run the configured test target (nearest test, file, or suite).	<code>neotest.lua</code>
<code>:TestSuite</code>	Run the configured test target (nearest test, file, or suite).	<code>neotest.lua</code>
<code>:TodoLocList</code>	Collect TODO-style comments in the current window's location list.	<code>todo-comments.lua</code>
<code>:TodoQuickFix</code>	Collect TODO-style comments in the quickfix list.	<code>todo-comments.lua</code>
<code>:TodoTelescope</code>	Search TODO-style comments in Telescope.	<code>todo-comments.lua</code>

Buffers & Windows

Extracted from `bufferlines.lua`.

Mode	Key	Action	Context
Normal	Shift-h	Previous buffer	Global
Normal	Shift-l	Next buffer	Global
Normal	Space bd	Close buffer	Global
Normal	Space bp	Pick buffer	Global

Dashboard

Extracted from alpha.lua.

Mode	Key	Action	Context
Normal	e	Start a new empty file	Dashboard buffer
Normal	ff	Find files	Dashboard buffer
Normal	fk	Find keymaps	Dashboard buffer
Normal	fp	Switch projects	Dashboard buffer
Normal	fr	Open recent files	Dashboard buffer
Normal	fs	Search project text	Dashboard buffer
Normal	lg	Open LazyGit	Dashboard buffer
Normal	nt	Toggle file explorer	Dashboard buffer
Normal	p	Open plugin manager	Dashboard buffer
Normal	q	Quit Neovim	Dashboard buffer
Normal	ss	Find and restore a session	Dashboard buffer

Debugging

Extracted from dap.lua.

Mode	Key	Action	Context
Normal	Space dB	Conditional breakpoint	Global
Normal	Space db	Toggle breakpoint	Global
Normal	Space dc	Continue	Global
Normal	Space de	Evaluate expression	Global
Normal	Space di	Step into	Global
Normal	Space dO	Step out	Global
Normal	Space do	Step over	Global
Normal	Space dp	Pause debug session	Global
Normal	Space dq	Terminate debug session	Global
Normal	Space dr	Restart debug session	Global
Normal	Space du	Toggle DAP UI	Global
Normal	Space dv	Toggle DAP virtual text	Global

Find & Navigate

Extracted from telescope.lua, todo-comments.lua.

Mode	Key	Action	Context
Normal	[t]	Previous todo comment	Global
Normal	]t	Next todo comment	Global
Normal	Space f/	Find in buffer	Global
Normal	Space fb	Find buffers	Global
Normal	Space fd	Find diagnostics	Global
Normal	Space ff	Find files	Global
Normal	Space fg	Find text	Global
Normal	Space fh	Find help	Global
Normal	Space fk	Find keymaps	Global
Normal	Space fL	Todo comments to location list	Global

Find & Navigate · part 2 of 2

Extracted from telescope.lua, todo-comments.lua.

Mode	Key	Action	Context
Normal	Space fp	Find project	Global
Normal	Space fR	Resume last picker	Global
Normal	Space fr	Find recent files	Global
Normal	Space fT	Todo comments to quickfix	Global
Normal	Space ft	Find todo comments	Global
Normal	Space fw	Find word under cursor	Global
Normal	Space gb	Git branches	Global
Normal	Space gc	Git commits	Global
Normal	Space gs	Git status	Global

Extracted from `gitsigns.lua`, `lazygit.lua`.

Mode	Key	Action	Context
Normal	[h]	Previous hunk	Global
Normal	]h	Next hunk	Global
Operator-pending	ih	Select hunk	Global
Visual	ih	Select hunk	Global
Normal	Space hB	Toggle line blame	Global
Normal	Space hb	Blame line	Global
Normal	Space hD	Diff against HEAD	Global
Normal	Space hd	Diff against index	Global
Normal	Space hi	Preview hunk inline	Global
Normal	Space hl	Hunks to location list	Global
Normal	Space hp	Preview hunk	Global
Normal	Space hQ	Repository hunks to quickfix	Global
Normal	Space hq	Hunks to quickfix	Global

Git · part 2 of 2

Extracted from `gitsigns.lua`, `lazygit.lua`.

Mode	Key	Action	Context
Normal	Space hR	Reset buffer	Global
Normal	Space hr	Reset hunk	Global
Visual	Space hr	Reset selection	Global
Normal	Space hS	Stage buffer	Global
Normal	Space hs	Stage hunk	Global
Visual	Space hs	Stage selection	Global
Normal	Space hU	Unstage buffer	Global
Normal	Space lG	LazyGit current file	Global
Normal	Space lg	LazyGit dashboard	Global
Normal	Space lL	LazyGit repository log	Global
Normal	Space ll	LazyGit file log	Global
Normal	Space uw	Toggle Git word diff	Global

LSP, Completion & Diagnostics

Extracted from lsp.lua.

Mode	Key	Action	Context
Normal	[d]	Previous diagnostic	Global
Normal	]d	Next diagnostic	Global
Normal	Space ld	Show diagnostic	Global
Insert	Ctrl-Space	Trigger LSP completion	LSP buffer (when attached)
Normal	gd	Go to definition	LSP buffer (when attached)
Normal	gi	Go to implementation	LSP buffer (when attached)
Normal	K	Show symbol information	LSP buffer (when attached)
Normal	Space la	LSP code action	LSP buffer (when attached)
Normal	Space lc	Toggle LSP completion	LSP buffer (when attached)
Normal	Space lf	Format buffer	LSP buffer (when attached)
Normal	Space li	Toggle inlay hints	LSP buffer (when attached)
Normal	Space lr	Refresh C# code lenses	LSP buffer (when attached)
Normal	Space lt	Run C# code lens	LSP buffer (when attached)

Markdown

Extracted from markdown.lua.

Mode	Key	Action	Context
Normal	Space ma	Toggle format on save	Global
Normal	Space mb	Browser preview (Mermaid)	Global
Normal	Space mc	Toggle task	Global
Visual	Space mc	Toggle task	Global
Normal	Space md	Lint Markdown	Global
Normal	Space mF	Unfold section	Global
Normal	Space mf	Fold section	Global
Normal	Space mh	Toggle hybrid preview	Global
Normal	Space mI	Follow link	Global
Normal	Space mN	Previous link	Global

Markdown · part 2 of 2

Extracted from markdown.lua.

Mode	Key	Action	Context
Normal	Space mn	Next link	Global
Normal	Space mp	Toggle preview	Global
Normal	Space ms	Toggle preview split	Global
Normal	Space mT	Toggle table mode	Global
Normal	Space mt	Format table	Global
Normal	Space mW	Add all project spelling words	Global
Normal	Space mw	Add project spelling word	Global
Normal	Enter	Mkdnflow smart Enter	Markdown buffer
Insert	Enter	Mkdnflow smart Enter	Markdown buffer
Visual	Enter	Mkdnflow smart Enter	Markdown buffer

New / uncategorized configuration

Extracted from mermaid_preview.lua.

Mode	Key	Action	Context
Normal	Space mD	Toggle Mermaid preview light/dark theme	Global
Normal	Space mv	Use M.browser_preview	Global

NvimTree

Extracted from nvimtree.lua.

Mode	Key	Action	Context
Normal	Space nc	Collapse nvim-tree	Global
Normal	Space ne	Expand nvim-tree	Global
Normal	Space nf	Reveal current file	Global
Normal	Space nn	Focus nvim-tree	Global
Normal	Space nr	Refresh nvim-tree	Global
Normal	Space nt	Toggle nvim-tree	Global

Personal Controls

Extracted from `remap.lua`, `which-key.lua`.

Mode	Key	Action	Context
Normal	<F2>	Toggle line numbers	Global
Normal	<F3>	Toggle relative line numbers	Global
Normal	Space ?	Buffer Local Keymaps (which-key)	Global
Normal	Space nh	Clear search highlights	Global

Testing

Extracted from `neotest.lua`.

Mode	Key	Action	Context
Normal	Space tA	Fallback: run test suite	Global
Normal	Space ta	Run test suite	Global
Normal	Space td	Debug nearest test	Global
Normal	Space tf	Run current file	Global
Normal	Space tn	Run nearest test	Global
Normal	Space to	Show test output	Global
Normal	Space ts	Toggle test summary	Global
Normal	Space tV	Fallback: run current file	Global
Normal	Space tv	Fallback: run nearest test	Global
Normal	Space tx	Stop test	Global

UI & Editing

Extracted from autopairs.lua, indent-blankline.lua.

Mode	Key	Action	Context
Normal	Space ua	Toggle autopairs	Global
Normal	Space ui	Toggle indent guides	Global
Normal	Space us	Toggle indent scope	Global

Workspaces & Sessions

Extracted from `auto-session.lua`.

Mode	Key	Action	Context
Normal	Space wr	Find session	Global
Normal	Space ws	Save session	Global